

SLIDE 1 — What is Searching in an Array?

Definition:

Searching means finding whether a particular value exists in an array.

Simple idea

Array → **Search Value** → **Compare** → **Found / Not Found**

Example:

```
int numbers[5] = {10, 20, 30, 40, 50};
```

Search for **30**:

10 → 20 → 30 
 ↑
 Found!

Explanation:

The program checks the elements one by one until it finds the required value.

 **Remember: Searching = Find a value**

SLIDE 2 — Linear Search

Linear search checks each element from the beginning to the end.

Example

```
int numbers[5] = {10, 20, 30, 40, 50};
```

```
int key = 30;
```

```
for (int i = 0; i < 5; i++) {  
    if (numbers[i] == key) {  
        printf("Found");  
    }  
}
```

Explanation:

numbers[0] → 10 ❌
numbers[1] → 20 ❌
numbers[2] → 30 ✅

Output:

Found

★ Remember

● Compare each element

● Match → Found

● No match → Not Found

SLIDE 3 — Finding the Position

Searching can also tell us the **index** where the value is found.

Example

```
int numbers[5] = {10, 20, 30, 40, 50};  
int key = 40;
```

```
for (int i = 0; i < 5; i++) {  
    if (numbers[i] == key) {  
        printf("Found at index %d", i);  
        break;  
    }  
}
```

Explanation:

Index:	0	1	2	3	4
	[10]	[20]	[30]	[40]	[50]
			↑		
			Found		

Output:

Found at index 3

 **Remember: Search → Compare → Get index**

SLIDE 4 — Found or Not Found

We can use a variable to track whether the value was found.

Example

```
int numbers[5] = {10, 20, 30, 40, 50};
int key = 60;
int found = 0;
```

```
for (int i = 0; i < 5; i++) {
    if (numbers[i] == key) {
        found = 1;
        break;
    }
}
```

```
if (found == 1)
    printf("Found");
else
    printf("Not Found");
```

Explanation:

```
Search 60
↓
Compare all elements
↓
No match ❌
↓
Not Found
```

Output:

Not Found

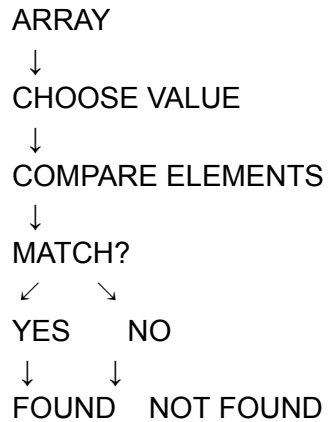
Basic Pattern

SEARCH → COMPARE → MATCH? → RESULT

SLIDE 5 — RECAP

Search and Find! 🔍

Flow



🎯 5 Things to Remember

1. Searching finds a value in an array
2. Linear search checks elements one by one
3. `==` is used to compare values
4. The index tells where the value is found
5. If no match exists → Not Found

👁️ Final Example

```
int numbers[5] = {10, 20, 30, 40, 50};
```

```
if (numbers[2] == 30)
    printf("Found");
```

30 == 30 → TRUE ✅

Output:

Found

⚡ Memory Trick

SEARCH → COMPARE 🔍

MATCH → FOUND ✅

NO MATCH → NOT FOUND ❌

QUESTIONS

1. What is the main purpose of searching an array?

- A) To delete elements
- B) To find a particular value
- C) To create an array
- D) To sort the array

💡 **Hint:** Searching means looking for something.

📖 **Glossary:** Search → Find a particular value.

✅ **Answer:** B

2. Which operator is used to compare the search value with an array element?

- A) =
- B) +
- C) ==
- D) !=

💡 **Hint:** We need the operator that checks equality.

📖 **Glossary:** Compare → Check whether two values are equal or different.

✅ **Answer:** C) ==

3. What is the output?

int numbers[5] = {10, 20, 30, 40, 50};

```
if (numbers[2] == 30)
    printf("Found");
```

 **Hint:** `numbers[2]` contains 30.

 **Glossary: Match** → When the searched value equals an array element.

 **Answer:** Found

4. Fill in the blank

Complete the searching condition:

```
for (int i = 0; i < 5; i++) {
    if (numbers[i] _____ key) {
        printf("Found");
    }
}
```

 **Hint:** Use the operator that checks whether two values are equal.

 **Glossary: Equality** → When two values have the same value.

 **Answer:** ==

5. If a value is not present in the array, what should the program display?

- A) Found
- B) Error
- C) Not Found
- D) Repeat

 **Hint:** There is no matching element.

 **Glossary: Not Found** → The searched value does not exist in the array.

 **Answer:** C) Not Found

6. What does the variable `i` usually represent during array searching?

- A) Array value
- B) Array index
- C) Array size
- D) Search result

 **Hint:** `i` moves from one position to another.

 **Glossary:** Index → Position of an element in an array.

 **Answer:** B) Array index

7. Arrange the steps in the correct order:

- A. Compare the value
- B. Choose a search value
- C. Display the result
- D. Access an array element

 **Hint:** Choose → Access → Compare → Result.

 **Glossary:** Result → The final outcome of the search.

 **Answer:** B → D → A → C